EduStack Masterclass

Your Ultimate Computer Science & Engineering Hub

VOLUME 4 — ML Microservice, FAANG System Design & Playbook

Tier-1 Interview Reference Guide (Visa, Amazon, Oracle, JPMC, Microsoft)

Python FastAPI | RAG Engine | Gemini Multimodal | 20 High-Scale Scenarios | Playbook

Developer:	Shubham Kumar | CSE Student | NIT Patna
Target Roles:	AI/ML Platform Engineer, Senior Systems Architect, SDE II
AI Engine:	Python 3.11 FastAPI + Google Gemini 1.5/2.0 + LightRAG Engine
PDF Multimodal:	Dual-Strategy Processing: pypdf Text Extraction + Gemini Vision OCR
Scenarios:	20 High-Scale FAANG System Design & Incident Defense Scenarios
Playbook:	2-Minute Architectural Elevator Pitch & Defense Framework
Volume 4:	Python ML Microservice, RAG Engine, 20 Scenarios & Master Playbook

github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub

For SDE Technical Interview Preparation — Volume 4 of 4

Table of Contents — Volume 4

SECTION 20

Python FastAPI ML Microservice Architecture

ASGI Uvicorn server, HTTP Proxy Gateway pattern, and language isolation

EduStack isolates its AI tutoring and document processing engine into a dedicated Python 3.11 FastAPI microservice. Node.js handles I/O-intensive web traffic, while Python manages CPU-intensive AI computations.

Python / FastAPI & System Design Solution

```
# ml_services/main.py  -  FastAPI Initialization & RAG Integration
from fastapi import FastAPI, HTTPException, UploadFile, File
import google.generativeai as genai
import os

app = FastAPI(title='EduStack AI Engine')
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

def get_available_gemini_models():
    try:
        models = [m.name for m in genai.list_models() if 'generateContent' in
m.supported_generation_methods]
        return models or ['models/gemini-1.5-flash', 'models/gemini-2.0-flash']
    except Exception:
        return ['models/gemini-1.5-flash', 'models/gemini-2.0-flash']
```

SECTION 21

RAG Engine & Gemini Integration

Retrieval Augmented Generation with LightRAG knowledge store and fallback loop

SYSTEM DESIGN MAP: Retrieval-Augmented Generation (RAG) Architecture

Phase 1: Phase 1: Question Input -> LightRAG Store retrieves top-3 relevant CS course chunks via TF-IDF

↓

Phase 2: Phase 2: Prompt Engine constructs augmented context prompt with course material

↓

Phase 3: Phase 3: Fallback Loop iterates Gemini models (gemini-2.0-flash -> gemini-1.5-flash)

↓

Phase 4: Phase 4: Gemini generates grounded factual response -> Returns answer + source references

SECTION 24

FAANG System Design & Incident Scenarios

20 Realistic, High-Impact Interview Scenarios with Full Technical Solutions

Scenario: Scenario 1: Distributed Botnet Login Attack Mitigation

Architectural Solution:

Implement a multi-tier sliding-window rate limiter using Redis. Tier 1: IP-based limit (10 req/min). Tier 2: Email account limit (5 failed logins per target email across ALL IPs in 15 min). After 5 failed attempts, temporarily lock the email in Redis (redis.setex("lock:"+email, 900, "1")). Always return the generic message "Invalid credentials".

-> IP-only limits fail against proxy botnets; email locking stops credential stuffing.

-> Atomic Redis Lua scripts prevent race conditions during high concurrency.

Scenario: Scenario 2: Zero-Downtime Password Hash Migration (bcrypt to Argon2id)

Architectural Solution:

Implement lazy re-hashing during active user logins. Add hashAlgorithm field to user schema (default "bcrypt"). During login: verify with bcrypt. If valid, asynchronously hash plaintext password with Argon2id, update MongoDB document, and set hashAlgorithm="argon2id". Inactive accounts remain securely hashed with bcrypt until next login.

-> Zero user disruption - re-hashing occurs transparently during login.

-> OWASP-recommended pattern for upgrading legacy password hash algorithms.